

TREKKING MANAGEMENT APPLICATION

PROJECT REPORT

STUDENT DETAILS

Name : Anish Bhattacharya
Roll Number : 23f2005522
Course Name : MAD1 Project
Email : 23f2005522@ds.study.iitm.ac.in

Brief Profile :

A learner pursuing the MAD1 course at IIT Madras , built full-stack web applications using Python and Flask previously worked with MERN stack. This project was developed as part of the course requirement to design and implement a role-based web application for managing trekking activities.

PROJECT DETAILS

This project is a web-based Trekking Management Application designed to simplify the management of trekking activities for adventure organizations. Trekkers can register, browse open treks, book slots, and view trekking history; Trek Staff can manage assigned treks and participants; and Administrators can manage treks, staff, trekkers, bookings, and reports — each role interacting with the system based on assigned permissions.

The application follows the MVC pattern and is built using Python, Flask, and SQLite, with a Bootstrap interface.

FRAMEWORKS AND LIBRARIES USED

- Backend: Python (Flask), Flask-SQLAlchemy (ORM), Flask-Migrate, Werkzeug
- Frontend: HTML, CSS, Jinja2, Bootstrap
- Database: SQLite
- Other Tools: Flask Blueprints, Flask Sessions

Why these?

Flask offers a minimal yet powerful web framework perfect for rapid development. SQLAlchemy simplifies database operations. Bootstrap ensures a responsive layout. Flask-Migrate supports database schema changes as the project grows. Werkzeug provides secure password hashing for user authentication. SQLite is ideal for lightweight, file-based storage during development.

DEMO VIDEO

To view the demonstration video
[LINK](#)

ER DIAGRAM

To view the ER diagram
[LINK](#)

DATABASE SCHEMA

- User: id, username, email, phone, password_hash, role, is_active, is_blacklisted, blacklisted_reason, created_at, updated_at
 - Staff: id, user_id (FK → User), joining_date, experience, address, contact_number, staff_bio, Profile_status, created_at, updated_at
 - Trek: id, name, location, difficulty, duration, total_slots, available_slots, price, description, image_url, starting_at, ending_at, status, assigned_staff_id (FK → Staff), created_at, updated_at
 - Booking: id, user_id (FK → User), trek_id (FK → Trek), booking_date, status, payment_status, amount_paid, booking_cancel_date, created_at, updated_at
-

DATABASE RELATIONSHIPS

user (1) ————— (1) staff === One user with role staff has one staff profile.
FK: staffs.user_id → users.id

staff (1) ————— (many) trek === One staff member can be assigned to many treks.
FK: treks.assigned_staff_id → staffs.id

user (1) ————— (many) booking === One trekker can have many bookings.
FK: bookings.user_id → users.id

trek (1) ————— (many) booking === One trek can have many bookings.
FK: bookings.trek_id → treks.id

SYSTEM ARCHITECTURE

___The application follows the MVC (Model-View-Controller) pattern ___

main.py	– Flask app entry point, blueprint registration, DB init
model/model.py	– SQLAlchemy models and enums
controller/	– Business logic (admin, auth, trekker, trek_staff)
routes/	– URL routing via Flask Blueprints
templates/	– Jinja2 HTML templates (admin, trekker, trekStaff)
utils/authentication.py	– Login and role-check decorators
db/seed_data.py	– Programmatic admin and dummy data seeding
instance/Trek.db	– SQLite database file
migrations/	– Flask-Migrate (Alembic) database migrations

FEATURES IMPLEMENTED

- User registration, login, logout with role-based dashboards (Admin, Trek Staff, Trekker)
- Admin can manage treks, staff, trekkers, bookings, search, and reports
- Trek Staff can manage assigned treks, participants, slots, and trek progress
- Trekkers can browse/filter open treks, book slots, cancel bookings, and view history
- Duplicate and overbooking prevention with trek/booking status tracking
- Responsive UI with Bootstrap across all role-specific pages

AI/LLM tools (including GitHub Copilot and ChatGPT) were used for approximately 15–22% of the development process.

The assistance was mainly for:

- *GitHub Copilot* : was used during this project, mostly for syntax correction, refactoring, and milestone mapping — that is, checking the requirements document to see if everything was completed according to the requirement document and if I had missed anything.

For Example — There was also a case where my own implementation did not match the requirement and then I refactored the code myself to remove staff self-registration and keep registration open only for Trekkers.

For Example — Before implementing each route controller, I first planned the feature's development flow by discussing the overall approach and validating my own design. Once the workflow was clear, I independently implemented the complete feature while following the planned architecture. Mainly in booking rules (open trek only, slot limits, duplicate prevention) , staff/Trekker approval/blacklist/reject flows and trek status lifecycle

For Example — I had missed error handling and try-except in a few routes. The copilot highlighted the inconsistency, and I corrected those parts on my own.

For Example — Also took help to separate out my routes and controllers (Manual work automated)

- *ChatGPT* : was used to draft HTML pages + bootstrap-classes with hardcoded dummy data for layout reference. After that, I reworked those pages on my own using Jinja2 templating (loops, conditionals, etc.) and kept the UI consistent across admin, trekker, and staff sections.

Apart from these, all core logic and features were implemented by me.

END OF REPORT
